

Numerikus Számítások

6. előadás

Csókás Eszter

Szegedi Tudományegyetem
Informatika Intézet
Számítógépes Optimalizálási Tanszék



Eliminációs mátrixok

Gauss-elimináció példa [bővebben a gyakorlatokon]

$$\begin{pmatrix} 1 & 1 & 1 \\ 2 & 3 & 1 \\ 1 & -1 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 6 \\ 10 \\ 7 \end{pmatrix}$$

$$\left(\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 2 & 3 & 1 & 10 \\ 1 & -1 & 2 & 7 \end{array} \right)$$

$$\left(\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 2 & 3 & 1 & 10 \\ 1 & -1 & 2 & 7 \end{array} \right) \xrightarrow{R_2 - 2R_1, R_3 - R_1} \left(\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 1 & -1 & -2 \\ 0 & -2 & 1 & 1 \end{array} \right) \xrightarrow{R_3 + 2R_2} \left(\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 1 & -1 & -2 \\ 0 & 0 & -1 & -3 \end{array} \right)$$

A kapott felső háromszög rendszer:

$$x_1 + x_2 + x_3 = 6$$

$$x_2 - x_3 = -2$$

$$-x_3 = -3$$

$$x_1 = 2$$

$$x_2 = 1$$

$$x_3 = 3$$

Eliminációs mátrixok - informálisan

- a trianguláris mátrix eléréséhez ekvivalens átalakításokat hatunk végre → ezeket az átalakításokat le lehet írni mátrixok segítségével!
- P_I : az egységmátrixszal való szorzás semmilyen átalakítást nem eredményez (*gondoljuk meg, hogy ennek mi az oka!*)
- de az egységmátrix kis változtatásával minden egyenletrendszeren végrehajtott művelet egyszerű mátrixszorzássá alakul
- ezeket, az átalakításra használt mátrixokat M -mel jelöljük és eliminációs mátrixoknak hívjuk

Eliminációs mátrixok

Olyan mátrixot viszonylag egyszerű megadni, amellyel balról való beszorzás azt eredményezi, hogy egy a vektor k -adik alatti együtthatói eltűnnek:

$$\mathbf{M}_k \mathbf{a} = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & & \ddots & \\ & & & & 1 \end{pmatrix} \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_k \\ a_{k+1} \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_k \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

Handwritten annotations: A red circle is around the 1 in the k -th row of the matrix. A blue box is around the sub-diagonal elements $-a_{k+1}/a_k, \dots, -a_n/a_k$. A red arrow points from the a_{k+1} element of the vector to the 0 in the $(k+1)$ -th row of the result vector. A red arrow points from the a_k element of the vector to the 0 in the $(k+1)$ -th row of the result vector.

Itt az $\mathbf{M}_k$ mátrixban minden ki nem írt együttható nulla, és az a_k együtthatót *generálóelemnek* hívjuk, magát az $\mathbf{M}_k$ mátrixot pedig *eliminációs mátrixnak*, vagy *Gauss-transzformációnak*.

Eliminációs mátrixok tulajdonságai

- $\mathbf{M}_k$ alsó háromszögmátrix, a főátlójában csupa egyessel, tehát reguláris (nem szinguláris) mátrix,
- $\mathbf{M}_k = \mathbf{I} - \mathbf{m}\mathbf{e}_k^T$, ahol $\mathbf{m} = (0, \dots, a_{k+1}/a_k, \dots, a_n/a_k)^T$, és $\mathbf{e}_k$ az egységmátrix k -adik oszlopa,
- $\mathbf{M}_k^{-1} = \mathbf{I} + \mathbf{m}\mathbf{e}_k^T$, azaz $\mathbf{M}_k$ és az inverze csak a főátlón kívüli elemek előjelében különböznek. Az $\mathbf{M}_k^{-1}$ alsó háromszögmátrixot $\mathbf{L}_k$ -val is fogjuk jelölni.
- Két eliminációs mátrix szorzatát könnyű megadni a $k < j$ esetre:

$$\mathbf{M}_k \mathbf{M}_j = (\mathbf{I} - \mathbf{m}\mathbf{e}_k^T)(\mathbf{I} - \mathbf{t}\mathbf{e}_j^T) = \mathbf{I} - \mathbf{m}\mathbf{e}_k^T - \mathbf{t}\mathbf{e}_j^T + \mathbf{m}\mathbf{e}_k^T \mathbf{t}\mathbf{e}_j^T = \mathbf{I} - \mathbf{m}\mathbf{e}_k^T - \mathbf{t}\mathbf{e}_j^T,$$

mivel $\mathbf{e}_k^T \mathbf{t} = 0$. Két eliminációs mátrix szorzata tehát a kettő kompozíciója a fenti képlet értelmében.

- Hasonlóan az inverzekre is érvényes $\mathbf{L}_k \mathbf{L}_j = \mathbf{I} + \mathbf{m}\mathbf{e}_k^T + \mathbf{t}\mathbf{e}_j^T$ (ha $k < j$).

Feladat #1

Legyen $\mathbf{x} = (1, 2, -3)^T$. Írjuk fel a hozzá tartozó eliminációs mátrixot!

$$\begin{pmatrix} 1 \\ 2 \\ -3 \end{pmatrix} \rightsquigarrow \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$$
$$M = \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 3 & 0 & 1 \end{pmatrix}$$

Feladat #2

Adjunk meg egy olyan eliminációs mátrixot, amely egy 3×3 -as egyenletrendszerben a 2. egyenlet háromszorosából kivonja a 3. egyenlet kétszeresét és hozzáadja az 1. egyenletet.

$$M = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 3 & -2 \\ 0 & 0 & 1 \end{bmatrix}$$

Gauss elimináció eliminációs mátrixokkal

Feladat

Tekintsük a következő egyszerű lineáris egyenletrendszert:

$$2x_1 + 4x_2 - 2x_3 = 2,$$

$$4x_1 + 9x_2 - 3x_3 = 8,$$

$$-2x_1 - 3x_2 + 7x_3 = 10.$$

Írjuk fel mátrix-vektor alakban!

$$\begin{pmatrix} 2 & 4 & -2 \\ 4 & 9 & -3 \\ -2 & -3 & 7 \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 2 \\ 8 \\ 10 \end{pmatrix}$$

$A \quad \cdot \quad \underline{x} = \underline{b}$

Eliminációs mátrixok

Informálisan: a trianguláris mátrix eléréséhez ekvivalens átalakításokat hatunk végre $\rightarrow$ ezeket az átalakításokat le lehet írni mátrixok segítségével!

Feladat: Írjunk fel egy eliminációs mátrixot, ami felső háromszög alakra hozza **A**-t:

$$A = \begin{pmatrix} 2 & 4 & -2 \\ 4 & 9 & -3 \\ -2 & -3 & 7 \end{pmatrix}$$

$$M_1 = \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}$$

$$M_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -1 & 1 \end{pmatrix}$$

$$\begin{aligned} &\cdot (R_2 - 2 \cdot R_1) \\ &\cdot (R_3 + R_1) \end{aligned}$$

$$M_1 \cdot A = \begin{pmatrix} 2 & 4 & -2 \\ 0 & 1 & 1 \\ 0 & 1 & 5 \end{pmatrix} \cdot (R_3 - R_2)$$

$$\begin{aligned} &M_2 \cdot M_1 \cdot A = \\ &= \begin{pmatrix} 2 & 4 & -2 \\ 0 & 1 & 1 \\ 0 & 0 & 4 \end{pmatrix} = U \end{aligned}$$

A lineáris egyenletrendszer megoldáshoz az $Ax = b$ egyenletrendszerből eliminációs mátrixokkal olyat szeretnénk kialakítani, amelynek baloldalán egy felső háromszögmátrix van, így a behelyettesítést végre tudjuk hajtani.

$$MAx = \underbrace{M_{n-1} \cdots M_1}_{\text{eliminációs mátrixok}} Ax = \underbrace{M_{n-1} \cdots M_1}_M b = Mb.$$

Itt M_1 az A mátrix a_{11} elemére, mint generáló elemre támaszkodik, M_2 az M_1A mátrix 2., főátlóbeli elemére stb.:

$$\begin{pmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{pmatrix} \xrightarrow{M_1} \begin{pmatrix} \times & \times & \times \\ 0 & x & x \\ 0 & x & x \end{pmatrix} \xrightarrow{M_2} \begin{pmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & x \end{pmatrix}$$

$A \qquad \qquad \qquad \underbrace{M_1A} \qquad \qquad \qquad \underbrace{M_2M_1A}$

Itt $\times$ az épp végrehajtott szorzás előtti mátrix elemeit jelöli, x pedig az utolsó lépésben megváltozott, nem feltétlen nulla együtthatókat.

Tehát, azt láttuk, hogy

$$\mathbf{MAx} = \mathbf{M}_{n-1} \cdots \mathbf{M}_1 \mathbf{Ax} = \mathbf{M}_{n-1} \cdots \mathbf{M}_1 \mathbf{b} = \mathbf{Mb}.$$

Szóval nem csak az $\mathbf{A}$ mátrixot szoroztuk be. Hajtsuk végre a $\mathbf{b}$ -n is az eliminációs mátrixokat!

$$\mathbf{M}_1 \mathbf{b} = \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 2 \\ 8 \\ 10 \end{pmatrix} = \begin{pmatrix} 2 \\ 4 \\ 12 \end{pmatrix}$$

$$\mathbf{M} \cdot \underline{\mathbf{Ax}} = \mathbf{M} \cdot \underline{\mathbf{b}}$$

$$\mathbf{M}_2 \mathbf{M}_1 \mathbf{b} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -1 & 1 \end{pmatrix} \cdot \begin{pmatrix} 2 \\ 4 \\ 12 \end{pmatrix} = \begin{pmatrix} 2 \\ 4 \\ 8 \end{pmatrix}$$

LU felbontás

$$A \in \mathbb{R}^{3 \times 3}$$

$$Ax = \underline{b}$$

$L \cdot U$ } háromszög alakú

$$M_2 \cdot M_1 \cdot A \cdot \underline{x} = M_2 \cdot M_1 \cdot \underline{b}$$

$$M_2 \cdot M_1 \cdot A = U \quad / M_2$$

$$A = LU$$

$$A = LU$$

$$\rightarrow M_2^{-1} M_2 \cdot M_1 A = M_2^{-1} \cdot U$$

$$I \quad M_1 \cdot A = M_2^{-1} \cdot U \quad / M_1^{-1}$$

$$A = \underbrace{M_1^{-1} \cdot M_2^{-1}}_L \cdot U$$

$$L_1 = M_1^{-1}$$

$$L_2 = M_2^{-1}$$

$$L = L_1 \cdot L_2 = M_1^{-1} \cdot M_2^{-1}$$

LU felbontás

$$\begin{pmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{pmatrix} \xrightarrow{\mathbf{M}_1} \begin{pmatrix} \times & \times & \times \\ 0 & x & x \\ 0 & x & x \end{pmatrix} \xrightarrow{\mathbf{M}_2} \begin{pmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & x \end{pmatrix}$$

$\mathbf{A}$ $\mathbf{M}_1\mathbf{A}$ $\mathbf{M}_2\mathbf{M}_1\mathbf{A}$

Itt $\times$ az épp végrehajtott szorzás előtti mátrix elemeit jelöli, x pedig az utolsó lépésben megváltozott, nem feltétlen nulla együtthatókat.

Vegyük észre, hogy ez az eljárás az eredeti $\mathbf{A}$ mátrixot két új, háromszögmátrix szorzatára bontja:
 $\mathbf{A} = \mathbf{L}\mathbf{U}$, ahol

$$\mathbf{L} = \mathbf{M}^{-1} = (\mathbf{M}_{n-1} \cdots \mathbf{M}_1)^{-1} = \mathbf{M}_1^{-1} \cdots \mathbf{M}_{n-1}^{-1} = \mathbf{L}_1 \cdots \mathbf{L}_{n-1}.$$

A konstrukcióból adódik, hogy $\mathbf{U} = \mathbf{M}\mathbf{A}$ egy felső háromszögmátrix, az $\mathbf{L}$ diagonális elemei pedig egyesek.

Az eredeti feladatunkat most az LU felbontás segítségével egyszerűsíthetjük: $\mathbf{Ax} = \mathbf{b}$ helyett vegyük az $\mathbf{LUx} = \mathbf{b}$ egyenletrendszert. Ezt két lépésben oldjuk meg, először az

$$\mathbf{Ly} = \mathbf{b}$$

egyenletrendszert az új, mesterséges $\mathbf{y}$ változóra, majd az

$$\mathbf{Ux} = \mathbf{y}$$

$\mathbf{Ax} = \mathbf{b}$
 $\downarrow \downarrow$
 $\mathbf{L} \cdot \mathbf{U}$
 $\mathbf{L} \cdot \mathbf{U} \cdot \mathbf{x} = \mathbf{b}$
 $\mathbf{y}$

egyenletet. vegyük észre, hogy $\mathbf{y} = \mathbf{Mb}$.

Az $\mathbf{Ax} = \mathbf{b}$ lineáris egyenletrendszer megoldására szolgáló Gauss elimináció formálisan a következő lépésekből áll:

1. Az $\mathbf{A}$ mátrix LU felbontása (egy alsó és egy felső háromszögmátrixra),
2. $\mathbf{Ly} = \mathbf{b}$ megoldása $\mathbf{y}$ -ra,
3. $\mathbf{Ux} = \mathbf{y}$ megoldása $\mathbf{x}$ -re.

1) $\mathbf{L} \cdot \mathbf{y} = \mathbf{b}$
2) $\mathbf{U} \cdot \mathbf{x} = \mathbf{y} \checkmark$

Vissza a példánkhoz, nézzük az $A = LU$ -t

Az LU felbontásban az L mátrix:

$$M_1 = \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} \rightarrow L_1 = M_1^{-1} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 0 & 1 \end{pmatrix}$$

$$M_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -1 & 1 \end{pmatrix} \rightarrow L_2 = M_2^{-1} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix}$$

$$L = L_1 L_2 = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 1 & 1 \end{pmatrix}$$

Vissza a példánkhoz, nézzük az $A = LU$ -t

Az LU felbontásban az L mátrix:

$$\mathbf{L} = \mathbf{L}_1 \mathbf{L}_2 = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 1 & 1 \end{pmatrix}.$$

Ezzel az $\mathbf{A}$ mátrix felbontása:

$$\mathbf{A} = \begin{pmatrix} 2 & 4 & -2 \\ 4 & 9 & -3 \\ -2 & -3 & 7 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 1 & 1 \end{pmatrix} \begin{pmatrix} 2 & 4 & -2 \\ 0 & 1 & 1 \\ 0 & 0 & 4 \end{pmatrix} = \mathbf{LU}.$$

 $L \cdot U$

MATLAB utasítások

A megfelelő Matlab program az LTriSol, UTriSol és az LU eljárásokra építve:

```
>> [L,U] = LU(A);      % LU felbontas  
>> y = LTriSol(L,b); % előre behelyettesites  
>> x = UTriSol(U,y); % hatra behelyettesites
```

A felhasznált LU eljárás Matlab kódja:

```
1 function [L, U] = LU(A)
2 [m,n] = size(A);
3 for k=1:n-1
4     A(k+1:n,k) = A(k+1:n,k)/A(k,k);
5     A(k+1:n,k+1:n) = A(k+1:n,k+1:n)-A(k+1:n,k)*A(k,k+1:n);
6 end
7 L = eye(n,n) + tril(A,-1);
8 U = triu(A);
```

Itt `tril` és `triu` az alsó-, illetve felső háromszögmátrixokat képezik az argumentum mátrixból kivágva. A második argumentum pedig a főátlótól való eltérés mértékét adja, tehát `tril(A, -1)` azt az alsó háromszögmátrixot adja, amelynek főátlójában is nullák vannak.

Miért jó nekünk az *LU* felbontás?

Az *LU*-felbontásnak több olyan előnyös tulajdonsága van, ami miatt használata meghatározó. Néhány a legfontosabbak közül:

- mivel az egyenletrendszer együtthatómátrixának *LU*-felbontásához **nincs szükség az egyenletrendszer jobb oldalára**, ezért használható olyan esetekben, amikor a jobb oldal még nem ismeretes, vagy több különböző jobb oldallal is dolgozni kell

Miért jó nekünk az LU felbontás?

Az LU -felbontásnak több olyan előnyös tulajdonsága van, ami miatt használata meghatározó. Néhány a legfontosabbak közül:

- mivel az egyenletrendszer együtthatómátrixának LU -felbontásához **nincs szükség az egyenletrendszer jobb oldalára**, ezért használható olyan esetekben, amikor a jobb oldal még nem ismeretes, vagy több különböző jobb oldallal is dolgozni kell
- az LU -felbontás ismeretében több mátrixokkal kapcsolatos számítás **gyorsabban elvégezhető** mint egyébként, pl. ilyen a mátrix inverzének, vagy determinánsának meghatározása

Miért jó nekünk az LU felbontás?

Az LU -felbontásnak több olyan előnyös tulajdonsága van, ami miatt használata meghatározó. Néhány a legfontosabbak közül:

- mivel az egyenletrendszer együtthatómátrixának LU -felbontásához **nincs szükség az egyenletrendszer jobb oldalára**, ezért használható olyan esetekben, amikor a jobb oldal még nem ismeretes, vagy több különböző jobb oldallal is dolgozni kell
- az LU -felbontás ismeretében több mátrixokkal kapcsolatos számítás **gyorsabban elvégezhető** mint egyébként, pl. ilyen a mátrix inverzének, vagy determinánsának meghatározása
- az LU -felbontás igen **memóriatakarékos**, ráadásul vannak olyan speciális mátrixosztályok (pl. a szalagmátrixok, vagy a ritka mátrixok), melyekre létezik a kiküszöbölésnél gyorsabb algoritmus az LU -felbontásra

Miért jó nekünk az LU felbontás?

Az LU -felbontásnak több olyan előnyös tulajdonsága van, ami miatt használata meghatározó. Néhány a legfontosabbak közül:

- mivel az egyenletrendszer együtthatómátrixának LU -felbontásához **nincs szükség az egyenletrendszer jobb oldalára**, ezért használható olyan esetekben, amikor a jobb oldal még nem ismeretes, vagy több különböző jobb oldallal is dolgozni kell
- az LU -felbontás ismeretében több mátrixokkal kapcsolatos számítás **gyorsabban elvégezhető** mint egyébként, pl. ilyen a mátrix inverzének, vagy determinánsának meghatározása
- az LU -felbontás igen **memóriatakarékos**, ráadásul vannak olyan speciális mátrixosztályok (pl. a szalagmátrixok, vagy a ritka mátrixok), melyekre létezik a kiküszöbölésnél gyorsabb algoritmus az LU -felbontásra
- a komputer algebra programok úgy működnek, hogy ha egy mátrixon valamilyen számítást kell elvégezni, ami megoldható az LU -felbontással (vagy a következőkben tárgyalandó PLU -felbontással), akkor azzal oldják meg, így ha később egy másik számítást is el kell e

Főelemkiválasztás

Lépcsős alak

Definíció: Egy mátrix lépcsős, vagy sorlépcsős alakú, ha kielégíti a következő két feltételt:

- a csupa 0-ból álló sorok (ha egyáltalán vannak) a mátrix utolsó sorai
- bármely két egymás után következő nem 0 sorban az alsó sor elején (legalább eggyel) több 0 van, mint a fölötte lévő sor elején

A nemnulla sorok első zérustól különböző elemének **főelem** (vezérelem vagy pivotelem), az ilyen elem oszlopának főoszlop (bázisoszlop) a neve.



4.1. Tétel. *Bármely mátrix elemi sorműveletekkel lépcsős alakra hozható.*

Redukált lépcsős alak

Definíció: Egy mátrix redukált lépcsős, vagy redukált sorlépcsős alakú, ha kielégíti a következő feltételeket:

- lépcsős alakú
- minden főelem egyenlő 1-gyel
- a főelemek oszlopaiban a főelemeken kívül minden elem 0

A főelemet itt vezéregyesnek vagy vezető egyesnek is szokás nevezni.

Például:

$$\begin{pmatrix} 0 & \boxed{1} \\ 0 & 0 \end{pmatrix}, \begin{pmatrix} \boxed{1} & 0 \\ 0 & 0 \end{pmatrix}, \begin{pmatrix} \boxed{1} & -2 & 0 & 4 \\ 0 & 0 & \boxed{1} & 6 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

4.2. Tétel. Minden mátrix redukált lépcsős alakra hozható, amely egyértelmű.

Főelemkiválasztás

Az LU felbontás csak akkor sikeres, ha az **A** mátrix nem szinguláris, és minden generáló elem nullától különböző. Ha a generálóelem (*főelem*, pivotelem) nulla, akkor még lehetséges lehet a felbontás – a mátrix átrendezése után, ami szintén ekvivalens feladathoz vezet. Ezt az eljárást *főelemkiválasztásnak* hívjuk.

Például az

$$\mathbf{A} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$


mátrix reguláris, mégse hajtható végre az LU felbontás közvetlenül. Szabad viszont a sorokat felcserélni (természetesen a lineáris egyenletrendszerben a jobboldali vektor elemei együtt mozognak).

Főelemkiválasztás

Tehát az $A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ mátrixon végezzük el a sorcserét.

A sorcserék utáni mátrix triviális LU felbontása:

$$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = LU,$$

illetve a sorok felcserélését elvégző P , ún. permutációs mátrixszal formálisan:

$$PA = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = LU,$$

→ a permutációs mátrix minden sorában és oszlopában egyetlen 1-es van, a többi elem 0

Amikor a főátló alatt az LU felbontás egy fázisában csak nulla elem van, akkor $\mathbf{M}_k = \mathbf{I}$ választással tovább folytathatjuk a felbontást.

Ha ez olyankor következett be, amikor a főátlóbeli elem nulla volt, akkor az így kapott LU felbontás nem lesz reguláris. Ez nem csoda, hiszen az eredeti $\mathbf{A}$ mátrix is szinguláris kellett hogy legyen.

Feladat: Határozzuk meg $\mathbf{A}$ LU felbontását, figyelembe véve a fentieket!

$$\mathbf{A} = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$$

Részleges főelemkiválasztás

A csak a sorcseréken alapuló főelemkiválasztást *részleges főelemkiválasztás*nak is nevezzük. Kicsit másként, részleges főelem-kiválasztásnak (részleges pivotálásnak) nevezzük a kiküszöbölési eljárások azon módját, melyben lebegőpontos adatokkal dolgozva minden oszlopban a szóba jöhető elemek közül - sorcserék segítségével - mindig a legnagyobb abszolút értékűt választjuk főelemnek. A hibaforrásoknál elmondottak miatt érdemes nagy abszolút értékű főelemeket választani.

Teljes főelemkiválasztás

Amikor a főelemet a teljes még szóba jövő részmátrixból választjuk, *teljes főelemkiválasztásról* beszélünk. Azaz, a teljes főelem-kiválasztás azt jelenti, hogy főelemnek nem csak a következő oszlopbeli, hanem az összes még hátralévő elem abszolút értékben legnagyobbikát választjuk. Bizonyos esetekben ezzel jobb eredményre jutunk, mint részleges főelem-kiválasztással, de műveletigényesebb, ritkán alkalmazzák.

Bár a permutáció elrontja az alsó- és felső háromszögmátrixokat, az egyenletrendszer megoldása szempontjából ezek teljes értékűek maradnak, így a jelölésben is szokás az eredeti LU alakot megtartani.

PLU-felbontás

Eddig azt láttuk, hogy nincs minden $\mathbf{A}$ mátrixnak LU -felbontása, de sorcserékkel – azaz egy permutáló mátrixszal való balról szorzással – olyan alakra hozható, melynek van LU -felbontása. Létezik tehát olyan $\mathbf{P}$ permutáló mátrix, hogy

$$\boxed{\mathbf{PA}} = \boxed{\mathbf{LU}}, \text{ azaz } \boxed{\mathbf{A} = \mathbf{P}^T \mathbf{LU}}.$$

(Handwritten red notes: $\mathbf{P}^{-1}$ with an arrow pointing to $\mathbf{P}^T$)

(Itt kihasználtuk, hogy permutáló mátrix inverze megegyezik transzponáltjával.)

Definíció: Egy tetszőleges $m \times n$ -es $\mathbf{A}$ mátrixnak egy permutáló, egy egység főátlójú négyzetes alsó háromszögös egy $m \times n$ -es felső háromszögmátrix szorzatára való bontását PLU-felbontásnak nevezzük.

A Matlab saját `lu` rutinja kezeli a részleges főelemkiválasztást, és a meghívási alakja függvényében különféle formában adja meg az eredményt:

```
>> A=[2 4 -2;4 9 -3;-2 -3 7];
```

```
>> [L,U,P]=lu(A)
```

L =

1.0000	0	0
-0.5000	1.0000	0
0.5000	-0.3333	1.0000

U =

4.0000	9.0000	-3.0000
0	1.5000	5.5000
0	0	1.3333

P =

0	1	0
0	0	1
1	0	0

Három kimeneti argumentummal tehát megkapjuk az L , U és P mátrixot is (utóbbi a permutációs mátrix). Ha csak két argumentumot adunk meg, akkor L permutált lesz:

>> $[L, U] = \text{lu}(A)$

$L =$

0.5000	-0.3333	1.0000
1.0000	0	0
-0.5000	1.0000	0

$U =$

4.0000	9.0000	-3.0000
0	1.5000	5.5000
0	0	1.3333

Ha nem adunk meg output argumentumot, akkor egyetlen eredmény mátrixot kapunk:

```
>> lu(A)
```

```
ans =
```

```
 4.0000    9.0000   -3.0000  
-0.5000    1.5000    5.5000  
 0.5000   -0.3333    1.3333
```

Ez a mátrix tömör formában tartalmazza az U felső háromszögmátrixot, és a permutálatlan M mátrixot. Utóbbi tehát a beszorzási együtthatókat adja (ezek az L elemeinek negáltjai). Ez a memóriatakarékos forma jellemző a számítógépes belső tárolásra. A sorvektor cseréket is csak egy segédvektoron szokás átvezetni, a sorok tényleges cseréje nem történik meg.

Műveletigény

Az LU felbontás műveletigénye közel $n^3/3$ szorzás és összeadás. A Matlab maga is mérte a végrehajtott lebegőpontos műveletek számát (a 6.5 változat már nem támogatja):

```
>> n = 100;  
>> flops(0); % Beallítja a számlálót 0-ra  
>> lu(rand(n));  
>> flops  
ans =  
    661650
```

Ez az érték megfelel a megadott $2/3 n^3$ összevont műveletigénynek. Ha a lineáris egyenletrendszer jobb oldalán egy egyszerű vektor van (nem több vektor, vagy mátrix), akkor a visszahelyettesítés további n^2 szorzást és összeadást igényel. Emiatt n növekedésével a lineáris egyenletmegoldás műveletigényének mind nagyobb része fordítódik az LU felbontásra.

CPU idő

A felhasznált CPU idő mérésére a következő rövid program használható:

```
>> n = 500;  
>> tic % indítja az orát  
>> lu(rand(n));  
>> toc  
elapsed_time =  
    0.4110
```

A kapott eredmény (700 Mhz-es Pentium III gépen) kb. 0.41 másodperc volt. Kétszeres méretű feladatra 2.5 másodperc kellett, ez összhangban van a számított műveletigénnyel (a $\text{rand}(n)$ $n \times n$ -es véletlen mátrixot állít elő egyenletes eloszlással).

Kondíciós szám

Mátrix inverz

Definíció: egy $n \times n$ -es négyzetes mátrix inverzének nevezzük azt az A^{-1} -gyel jelölt mátrixot, melyre $A \cdot A^{-1} = A^{-1} \cdot A = I$, ahol I a megfelelő méretű egységmátrix. (A definíció alapján egyértelmű, hogy az inverz mérete megegyezik az eredeti mátrix méretével.)

Kiszámítási lehetőségek: több módszer létezik, ezek közül egy ismert eljárás az adjungált aldetermináns kiszámításán alapszik, míg egy másik a Gauss-Jordan-elimináción. (Az óra végén ez utóbbira látunk majd példát.)

Reguláris, szinguláris mátrixok

Reguláris mátrix: $n \times n$ -es mátrixra a következők ekvivalensek:

- $\mathbf{A}$ reguláris, $\det(\mathbf{A}) \neq 0$
- $\mathbf{A}^{-1}$ létezik
- $\mathbf{Ax} = \mathbf{b}$ egyértelműen megoldható bármely $\mathbf{b} \in \mathcal{R}^n$ vektorra
- $\mathbf{Ax} = \mathbf{0}$ homogén rendszer egyetlen megoldása az $\mathbf{x} = \mathbf{0}$
- (az $\mathbf{A}$ sorvektorai (oszlopvektorai) lineárisan függetlenek)

Szinguláris mátrix: $n \times n$ -es mátrixra a következők ekvivalensek:

- $\mathbf{A}$ szinguláris, $\det(\mathbf{A}) = 0$
- $\mathbf{A}^{-1}$ NEM létezik
- $\mathbf{Ax} = \mathbf{b}$ egyenletrendszernek vagy végtelen sok megoldása van vagy nincs megoldása
- $\mathbf{Ax} = \mathbf{0}$ homogén lineáris egyenletrendszernek végtelen sok megoldása van
- (az $\mathbf{A}$ sorvektorai (oszlopvektorai) lineárisan függő)

Kondíciós szám

A numerikus számítás szempontjából döntő kérdés, hogy a korábbi, esetleg öröklött hibák milyen relatív hibához vezetnek a számítás során. Ahogy majd látni fogjuk, bizonyos feladatok esetén a relatív hibák növekedése elkerülhetetlen (lsd előző példa is!) → ezek *rosszul kondicionált* (ill-conditioned) feladatok

Egy $f(x)$ kiszámítása során (függetlenül a számítás módjától) a relatív hiba kb. ezerszerese lesz az input hibájának. A rosszul kondicionáltság mérésére szolgál, differenciálható kifejezések esetén, a *kondíciós szám*, mely a κ .

κ = a relatív hiba (aszimptotikus) növekedésének mértéke

Nézzük kicsit kézzelfoghatóbban...

Kondíciós szám

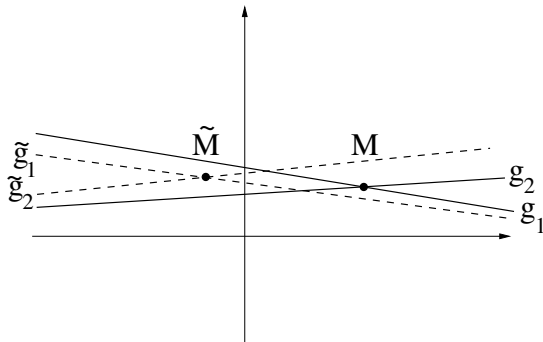
Tetszőleges reguláris $\mathbf{T}$ mátrix esetén a $\text{cond}(\mathbf{T}) := \|\mathbf{T}\| \|\mathbf{T}^{-1}\| \in \mathcal{R}^+$ számot $\mathbf{T}$ ($\|\cdot\|$ normára vonatkozó) *kondíciós számának* nevezzük. Szinguláris $\mathbf{T}$ -re $\text{cond}(\mathbf{T}) := \infty$.

Ha az $\mathbf{Ax} = \mathbf{b}$ lineáris egyenletrendszert tekintjük, akkor $\text{cond}(\mathbf{A})$ egy korlátot ad arra, hogy mennyire lesz az $\mathbf{x}$ megoldás pontatlan. Ez a korlát már azelőtt rendelkezésre áll, hogy bármilyen algoritmust végrehajtunk. A mátrixok kondíciószáma a mátrix tulajdonsága, és nem egy algoritmusé, vagy nem a számítógépen használt lebegőpontos számrendszertől függ! A kondíciószámra általában így gondolunk: ha $\mathbf{b}$ változik, akkor mennyire fog $\mathbf{x}$ változni?

- Ha $\text{cond}(\mathbf{A})$ nagy, akkor egy kicsi változás a $\mathbf{b}$ vektorban nagyméretű hibához vezet az $\mathbf{x}$ vektorban,
- ha viszont $\text{cond}(\mathbf{A})$ kicsi, akkor az $\mathbf{x}$ vektor hibája nem lesz sokkal nagyobb, mint a $\mathbf{b}$ vektor hibája.

Kondíciós szám - geometria

Két egyenlet 2 darab egyenest határoz meg, ezek metszéspontjait keressük geometriailag.



A kondíciós szám tulajdonságai:

- $\text{cond}(\mathbf{I}) = 1$, tehát az egységmátrix kondíciós száma egy¹
- $\text{cond}(\mathbf{A}) \geq 1$
- $\text{cond}(\mathbf{A}) = \text{cond}(\mathbf{A}^{-1})$
- ha $c \neq 0$, akkor $\text{cond}(c\mathbf{A}) = \text{cond}(\mathbf{A})$
- a $\mathbf{P}$ permutációs mátrixokra $\text{cond}(\mathbf{P}) = 1$
- a $\mathbf{D}$ diagonális mátrixokra $\text{cond}(\mathbf{D}) = \max |d_{ii}| / \min |d_{ii}|$
- $\text{cond}(\mathbf{AB}) \leq \text{cond}(\mathbf{A}) \text{cond}(\mathbf{B})$
- $\text{cond}_{\infty}(\mathbf{A}) = \text{cond}_1(\mathbf{A}^T)$
- $\text{cond}_F(\mathbf{A}) = \text{cond}_F(\mathbf{A}^T)$

¹ha a norma vektornormából származtatott

Feladat:

Határozzuk meg az $\mathbf{A} = \begin{pmatrix} 2 & 9 \\ 1 & 5 \end{pmatrix}$ mátrix kondíciós számát úgy, hogy az 1-es mátrixnormát használjuk.